

Lab 4 Document

Scenario

The objective of this exercise is to perform Create, Update, and Delete operations on Employee data using ASP.NET Core Web API.

Objective

- Update employee information using the HTTP PUT method.
- Delete employee information using the HTTP DELETE method.
- Use the **FromBody** attribute to receive employee data.
- Validate employee IDs before performing update and delete operations.
- Test the API using Swagger.

Project Name

FirstWebAPI

Implementation

Step 1: Open Existing Project

Opened the existing **FirstWebAPI** project.

Step 2: Modify EmployeeController

Updated the PUT action method to accept Employee data using the **FromBody** attribute.

The method validates the employee ID before updating the record.

If the employee ID is less than or equal to zero, or if the employee does not exist, the API returns:

Invalid employee id

If the employee ID is valid, the employee information is updated and returned as the response.

Step 3: Implement DELETE Method

Added a DELETE action method to remove an employee based on the employee ID.

The method validates the employee ID before deleting the record.

If the employee ID is invalid, the API returns:

Invalid employee id

Otherwise, the employee record is removed successfully.

Step 4: Build the Project

Built the project successfully using:

Build → Rebuild Solution

Step 5: Execute the Application

Executed the application using:

Ctrl + F5

Swagger UI opened successfully.

Step 6: Test PUT Method

Executed the **PUT /api/Employee/{id}** endpoint.

Provided employee information in JSON format using the request body.

The employee details were updated successfully and returned in the response.

Step 7: Test Invalid Employee ID

Executed the PUT request using invalid employee IDs.

The API returned:

400 Bad Request

Invalid employee id

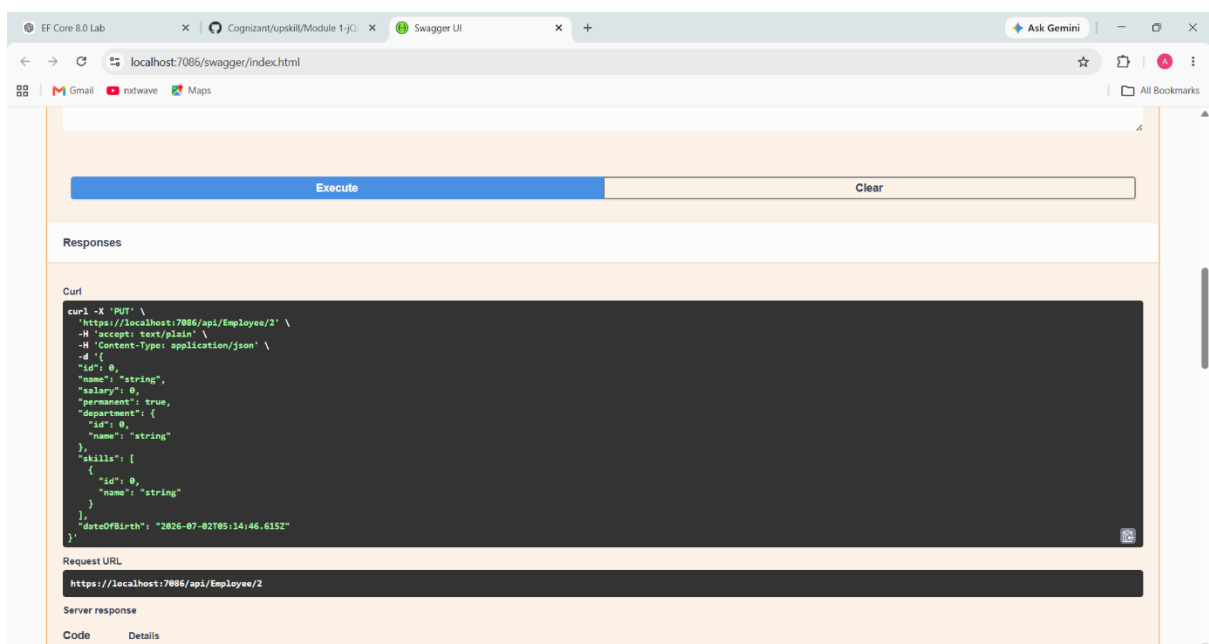
Step 8: Test DELETE Method

Executed the **DELETE /api/Employee/{id}** endpoint.

The employee record was deleted successfully.

Result

The ASP.NET Core Web API successfully performed **Update** and **Delete** operations using the HTTP **PUT** and **DELETE** action methods. The **FromBody** attribute was used to map JSON request data to the Employee model. Employee IDs were validated before performing update and delete operations, and appropriate **BadRequest** responses were returned for invalid requests. All CRUD operations were verified successfully using Swagger.



EF Core 8.0 Lab | Cognizant/upskill/Module 1-jc | Swagger UI | Ask Gemini

localhost:7086/swagger/index.html

GET /api/Employee/standard

PUT /api/Employee/{id}

Parameters

Cancel Reset

Name	Description
id * required	
integer(\$int32)	2
(path)	

Request body

application/json

```
{  "id": 0,  "name": "string",  "salary": 0,  "permanent": true,  "department": {    "id": 0,    "name": "string"  },  "skills": [    {      "id": 0,      "name": "string"    }  ],  "dateOfBirth": "2026-07-02T05:14:46.615Z"}
```

EF Core 8.0 Lab | Cognizant/upskill/Module 1-jc | Swagger UI | Ask Gemini

localhost:7086/swagger/index.html

Name Description

id * required

integer(\$int32)

0

(path)

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \  'https://localhost:7086/api/Values/0' \  -H 'accept: */*'
```

Request URL

https://localhost:7086/api/Values/0

Server response

Code	Details
200	Response body Record Deleted Successfully Download

Response headers

EF Core 8.0 Lab

Cognizant/upskill/Module 1-JC

Swagger UI

Ask Gemini

localhost:7086/swagger/index.html

Gmail

outwave

Maps

All Bookmarks

Values

GET

/api/Values

POST

/api/Values

GET

/api/Values/{id}

PUT

/api/Values/{id}

DELETE

/api/Values/{id}

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	0
(path)	

Execute

Clear

Responses